NOTES · PART 3

CLEAN OUTPUT AND PRODUCTION AGENTS

Memorandum on Agentic AI Applications · 24 Oct 2026 · continues Part 2

Contents

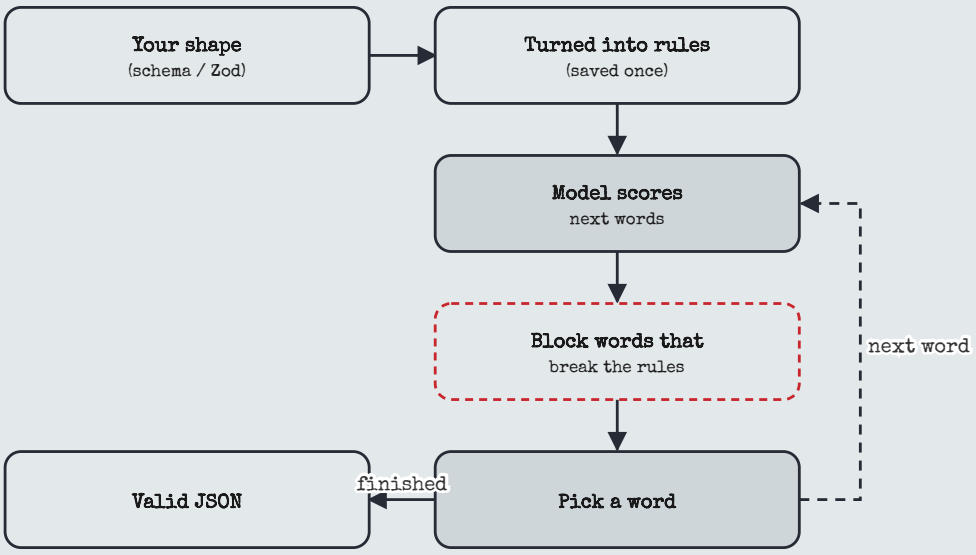
XII. Inside clean output and streaming

XIII. Inside a production agent

Diagram key: a shaded box is the AI model or agent, a red dashed box is a risk or a guard, plain boxes are people, tools, data and results. A dashed arrow means "go back and repeat".

XII. INSIDE CLEAN OUTPUT AND STREAMING

1. Why the fixed-form answer cannot break



Structured output: words that would break the shape are blocked before the model can pick them.

1. Your shape is turned into a set of rules (a state machine or grammar). This happens once and is saved, so the first call is a little slower.
2. At every word, the rules list which words are allowed next. All others get a chance of zero.
3. The model picks only from the allowed words, so the output always fits the shape.
4. Result from OpenAI's launch: 100% on their hard shape test, against under 40% for an older model without it.

- The shape is guaranteed. The values are not. Keep checking them in code (prices, totals, IDs).

2. The order of streaming events

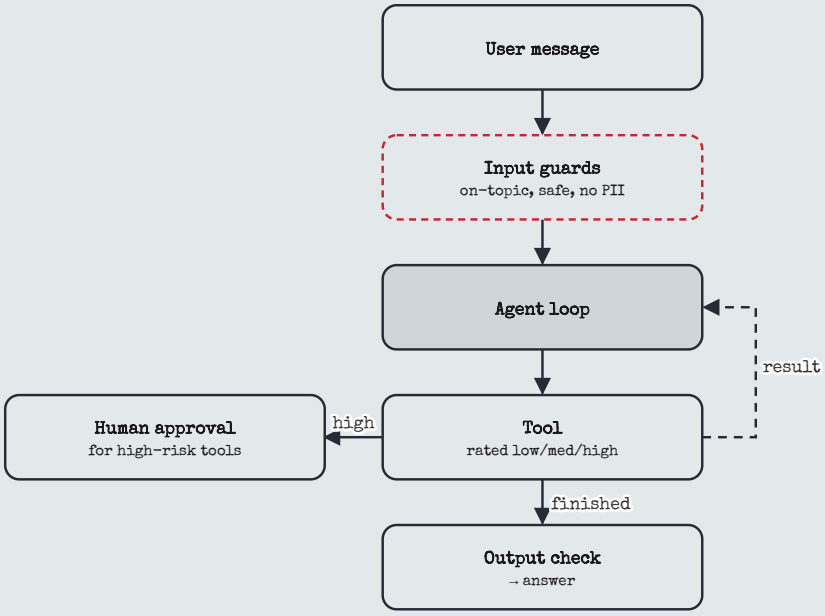
EVENT	WHAT IT CARRIES
message_start	an empty message
content_block_start	a new block begins (text or tool call)
content_block_delta	a small piece: text, part of a tool's JSON, or thinking
content_block_stop	that block is done
message_delta	why it stopped, and the token count
message_stop	all done (errors can also arrive mid-stream)

- Show text pieces as they arrive. Only act (save, pay, send) on the finished message.

> READ THE PAPER
>
> ▸ Introducing Structured Outputs (OpenAI): strict mode and the 100% result
>
> ▸ Efficient Guided Generation (Willard & Louf, 2023): the blocking method inside
>
> ▸ Streaming messages (Anthropic docs): every event type

XIII. INSIDE A PRODUCTION AGENT

1. Guards in layers



No single guard is enough, so stack them.

- The loop ends when the model gives a final answer, calls no tool, hits an error, or reaches the turn limit.
- Start with one agent. Split into several only when the instructions get too tangled or the tools overlap.

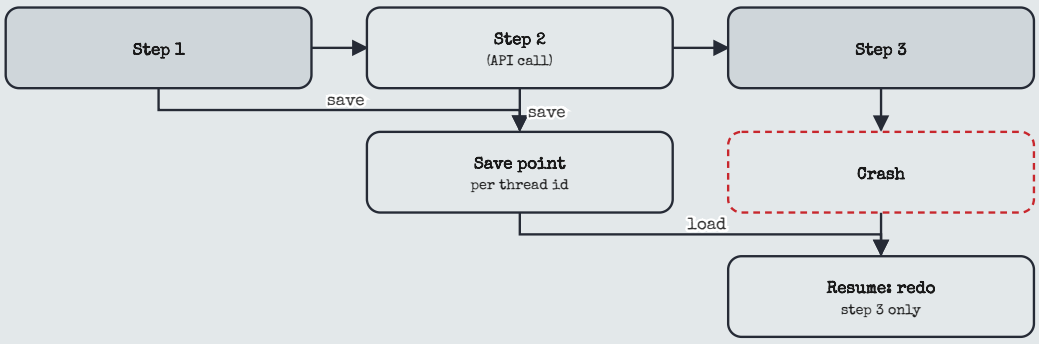
2. The 12 factors on one page

#	FACTOR	IN PLAIN WORDS
1	Natural language to tool calls	The model turns words into a structured action
2	Own your prompts	No hidden prompts from a framework
3	Own your context window	You decide what goes in, in what format
4	Tools are structured outputs	A tool call is just JSON your code acts on
5	One state	Agent progress and business data live together
6	Start, pause, resume	Simple calls to control a run
7	Contact humans with tools	"Ask for approval" is just another tool
8	Own your control flow	Your code decides what happens next
9	Compact errors	Put short error notes back in, with a retry limit
10	Small, focused agents	3 to 10 steps each, not 100
11	Trigger from anywhere	Slack, email, timer, webhook
12	Stateless reducer	(state, event) → new state

3. How pause-and-ask really works

1. The agent reaches a risky step and calls `interrupt(...)` with what it wants to do.
2. Everything is saved under a thread id (a save slot), and the run stops.
3. Your app shows the request. The person approves, edits or rejects.
4. You resume the same thread id with their answer.
5. Watch out: the paused step starts again from its top, so anything before the pause must be safe to run twice.

4. How long jobs survive a crash



Save after every step; after a crash only the unfinished step runs again.

- Save modes: at the end only (fast, risky), in the background, or before every next step (safest).
- Put side effects (emails, payments) in their own saved step and give them an idempotency key, so a retry never sends twice.

> READ THE GUIDE
>
> ▸ A Practical Guide to Building Agents (OpenAI, PDF): guards and when to split agents
>
> ▸ 12-Factor Agents (HumanLayer): the 12 rules
>
> ▸ Human-in-the-loop (LangGraph docs): interrupt and resume
>
> ▸ Durable execution (LangGraph docs): save points and save modes

* * *

Continued in Part 4.